



PERGAMON

Information Systems 27 (2002) 411–424



www.elsevier.com/locate/infosys

Agility through coordination

Luís Filipe Andrade^{a,*}, José Luiz Fiadeiro^{a,b}

^a *ATX Software S.A., Alameda António Sérgio 7-1C, 2795-023 Linda-a-Velha, Portugal*

^b *Department of Informatics, Faculty of Sciences, University of Lisbon, Campo Grande, 1749-016 Lisbon, Portugal*

Abstract

Information System Engineering has become under increasing pressure to come up with software solutions that endow systems with the agility that is required to evolve in a continually changing business and technological environment. In this paper, we suggest that Software Engineering has a contribution to make in terms of concepts and techniques that have been recently developed for Parallel Program Design and Software Architectures. We show how such mechanisms can be encapsulated in a new modelling primitive—coordination contract—that can be used for extending Component-Based Development approaches in order to manage such levels of change. © 2002 Elsevier Science Ltd. All rights reserved.

1. Introduction

More and more companies live in a very volatile and turbulent environment in which both business rules and supporting technology change very quickly. In order to remain competitive, companies need their information systems to be easily adaptable to such changes. And, most of the time, the required adaptations must be performed in a way that does not imply interruptions to the provision of services that are critical. Through the advent of the Internet and Wireless Applications, the New Economy is only fuelling this need even further: “...the ability to change is now more important than the ability to create e-commerce systems in the first place. Change becomes a first-class design goal and requires business and technology architecture whose components can

be added, modified, replaced and reconfigured” [1].

Component-Based Development (CBD) [2] has been often proclaimed to be the ultimate approach for providing software solutions with the *agility* required to cope with such turbulence in business and technological environments. As put in [3], “Software developers have long held the belief that complex systems can be built from smaller components, bound together by software that creates the unique behaviour and forms of the system. Ideally, a new system can be built using mostly predefined parts, with only a small number of new components required... In a well-designed system, the changes will be localised, and the changes can be made to the system with little or no effect on the remaining components”.

However, it is also widely recognised that the promises brought in by CBD are not without several caveats. Our own experience in developing applications in one of the most volatile business areas—banking—indicates that *interactions* and

*Corresponding author.

E-mail addresses: landrade@atxsoftware.com (L.F. Andrade), jose@fiadeiro.org (J.L. Fiadeiro).

architectures are at the core of the problems that still need to be addressed before CBD can deliver the goods. Many of the changes required on a system do not concern the computations performed by its components but the way they *interact*. Indeed, we often forget that the global behaviour of a system emerges both from the local behaviour of its components *and* the way they are interconnected. In dynamic business areas, the most frequent changes are likely to occur not at the level of the core entities of the domain (say, the notion of a bank account) but at the level of the business rules that determine how these core entities interact (say, how customers interact with their accounts). The old-fashioned technique of coding interactions in the components that implement the core services (say, reflecting policies on cash withdrawals directly on the withdrawal operation of an account) leads to systems that are very difficult to evolve because any change on the business rules triggers changes on the core components that it involves, their clients, possibly their client's clients, and so on.

Notice that object-oriented methods are very old-fashioned in this respect: interactions between objects are usually coded in the way messages are passed, features are called, and objects are composed, leading to intricate spaghetti-like structures (request threads) that are difficult to understand, let alone change. Hence, new methodological concepts and supporting technology are needed that promote interactions to first-class entities, leading to systems that are “exoskeletal” in the sense that they exhibit their configuration structure explicitly [4]. Such an externalisation of the interconnections should allow for systems to be reconfigured, in a compositional, non-intrusive way, by acting directly on the entities that capture the interactions between components, leading to an evolution process that follows the *architecture* of the system [5,6].

Whereas interactions relate two or more components, the technology that is required for externalising them is also meaningful when applied to the evolution of single components. Indeed, another class of changes often required on a system concerns the adaptation of individual components [7], either existing ones (as in legacy

systems) or newly acquired ones (as from third-party suppliers). In both cases, the implementation of the components is often not available and new ones cannot be brought in to replace them. A solution is to connect each component that needs to be evolved to an adapter such that the behaviour that emerges from the interaction between component and adapter fulfils the changes that were required initially. The advantage of making this interconnection explicit is that the adapter can be evolved independently of the component that it regulates, thus adding flexibility with respect to future changes.

In order to cope with the complexity of system evolution, we need more than mechanisms for managing change: we need to be able to determine *what* needs to be changed. Because change is most easily perceived and understood at the level of the application domain, the component model must be abstract enough to accommodate these changes without bringing in implementation details. This requires that the more traditional low-level notion of software component, and the design techniques used for its development (e.g., design patterns [8]), be “upgraded” to account for entities that are meaningful at the level of the application domain. This reflects the principle of “business centrality” upheld in [9] according to which software should be architected such that it reflects the way business organisations, policies and processes are defined by their own people. The ultimate goal of this approach is “100% maintenance” by which the authors mean “continuous quick evolution performed directly by the user rather than by the producer of the system”.

This “isomorphism” between business component and software component architecture requires that changes perceived at the level of the business domain be mapped to the implementation levels in a compositional, non-intrusive way by keeping them local to the implementations of the higher-level parts that need to be changed. In summary, the architectural structure of the system—its gross decomposition in terms of high-level components and the interconnections that bind them together—should reflect the structure of the business domain, and mechanisms should be provided that enable evolution to be compositional over this

structure, i.e. the evolution of the whole should be obtained from the evolution of its parts.

In this paper, we argue that the entities that provide interconnections between components and the adapters of single components, all of which are called *coordination contracts* in the rest of the paper, should be seen as *coordination* mechanisms that one may *superpose*, in a non-intrusive way, on existing systems, supporting “plug-and-play” in system construction and evolution. The terms “coordination” and “superposition” in the previous sentence have a precise technical meaning and they constitute the kernel of the contribution that we think Software Engineering can make to enhancing the *agility* of information systems.

Through “coordination” we mean principles and techniques that have been recently developed for separating what in systems accounts for the computations that are performed and the interactions that are maintained [10]. Our belief is that CBD needs to be enhanced with primitives that provide for the explicit representation of the coordination mechanisms that regulate the way components behave and interact, and enable systems to evolve through the reconfiguration of the coordination mechanisms in place, without having to interfere with the way the computations performed by the individual components are programmed.

The term “superposition” refers to a mechanism, also called “superimposition” by some authors, that has been developed and applied in Parallel Program Design for extending systems while preserving their properties [11–14]. Our belief is that this basic mechanism is lacking in object-oriented development approaches and fills the gap that we have identified in OO development for supporting the externalisation of interactions and their evolution.

These are the two sides of the coin that we call *Coordination Technologies* [15] and with which we want to contribute to the debate that has been launched for identifying synergies between Software and Information System Engineering. In Section 2, we provide further motivation on what these Coordination Technologies are and how they can contribute to increased levels of agility in the next generations of information systems. In Sec-

tion 3, we discuss an extension that we proposed in [16] for the UML in terms of a new semantic primitive that encapsulates Coordination Technology—coordination contracts or, simply, *contracts*. In Section 4, we discuss the semantics and deployment of contracts. We conclude with an overview of what is available on contracts and what we are planning to make available in the near future.

2. Motivation

In order to motivate the principles and techniques that we wish to put forward through coordination contracts, namely the difference to what is currently available through object-oriented modelling, consider the familiar world of bank accounts and customers who can make deposits, withdrawals, transfers, and so on.

In a typical classroom modelling exercise, a class account is defined with, among others, an operation allowing for the current balance to be obtained, and an operation *withdrawal* with parameter *amount*. A customer that owns an account is also part of the business domain and, as such, should figure explicitly, under some representation, in the model with an attribute *owns:account* so that transactions performed by that customer can be modelled through calls of the form *owns.withdrawal(amount)*. The condition $\text{balance} \geq \text{amount}$ is typically declared on *account* as a precondition on *withdrawal(amount)* to ensure that there are enough funds for the withdrawal to be made. This precondition establishes a contractual relationship between *account* and *customer*, both in the technical sense developed by B. Meyer [17] and as one of the business rules that establish the way customers are required to interact with accounts (Fig. 1).

This precondition derives more from the specification of a business requirement than an intrinsic constraint on the functionality of a basic business entity like *account*. Market evolution has shown that the circumstances under which a withdrawal can be accepted keeps changing as competition dictates banks to come up with new ways for customers to interact with their accounts.

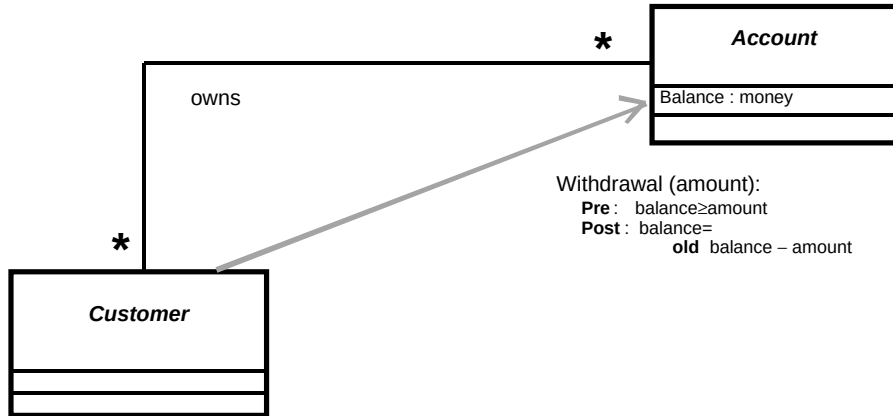


Fig. 1. Customers are clients of Accounts.

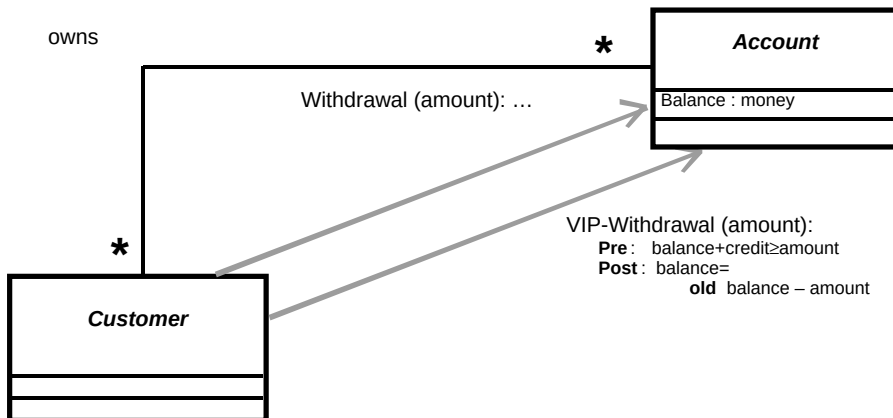


Fig. 2. Extending the interface of account with a new method.

Different financial packages are offered every now and then that provide different policies on withdrawals, making it impossible to establish once and for all how accounts will be accessed over their lifetime. Nevertheless, there are basic functional properties that should remain much more stable, reflecting the core “semantics” of the operation as a business transaction, like the fact that the amount is actually deducted from the balance.

In order to understand the problems raised by the need to reconfigure systems as a reaction to the constant evolution of market rules, assume that it becomes necessary to make certain instances of the interaction between customers and accounts more

flexible by creating “VIP-packages” according to which accounts can be overdrawn up to an amount agreed in advance between the customer and the bank. A naive solution to the problem of adapting the existing system to the new business rule would be to enrich account with a new operation—VIP-withdrawal—for the more flexible withdrawals (Fig. 2). Besides requiring obvious and direct changes to the class account, this solution is also intrusive on the client side because customers now have to decide on which operation to call. A further disadvantage of this solution is in the fact that the “business rule”, i.e. the VIP status of the relationship between a customer and an

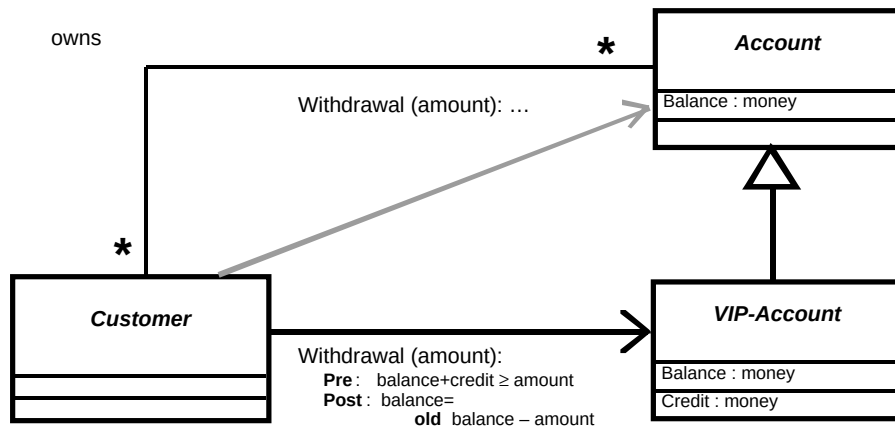


Fig. 3. Using inheritance to specialise the service.

account, is completely coded in the way the customer calls the account and, thus, cannot be “managed” explicitly as a business notion.

The typical OO-solution to this new situation is different: it consists in defining a subclass **VIP-account** of **account** with a new attribute **credit** and a weaker precondition on **withdrawal (amount): balance+credit ≥ amount** (Fig. 3). In this way, the more flexible contractual relationships can be established directly between the client (customer) and the specialisation of the supplier (the **VIP-accounts**). Nevertheless, there are two main drawbacks in this solution. On the one hand, it introduces, in the conceptual model, classes that have no counterpart in the real problem domain. It is the customers who are **VIPs**, not the accounts. However, having placed the contractual relationship between customers and accounts in **account**, one is forced to model the revised contract through a specialisation of the previous one, which implies the definition of the artificial subclass of **account**. The second disadvantage is not methodological but a technical one. The new solution is still intrusive because the other classes in the system need to be made aware of the existence of the new specialised class so that links between instances can be established through the new class. More generally, references to accounts need to be reclassified between “regular” and **VIP-accounts**.

The main disadvantages in both solutions result from the fact that the contract between customers and accounts is being placed on the side of the

supplier (the account), which is what is favoured by Meyer’s notion of contract [17]. This makes it difficult to accommodate new forms of the contract that depend on the client side of it. Hence, it makes more sense to place the contract at the level of the relationship that exists between customers and accounts by promoting it to what in the UML is known as an *Association Class*, i.e. an association that can also have class properties and features. In the account package example, the desired promotion could be achieved by introducing an association class **ownership** in which the details that pertain to the envisaged coordination of the joint behaviour of customers and accounts, including the preconditions that apply to the interactions, can be placed (Fig. 4).

Changes to the ownership contract, such as the addition of an attribute **credit** and the weaker precondition on the interaction, can now be put on specialisations of the new class without having to change the role classes. Indeed, resorting to association classes keeps the model faithful to the application domain by representing explicitly the business rules that coordinate the interaction between the entities involved. As a consequence, changes in the business rules should be more easily accommodated in the model without having to modify the classes that model the entities involved and, hence, leave a vast core of the information system unchanged. However, association classes still present severe drawbacks as a solution to our problem.

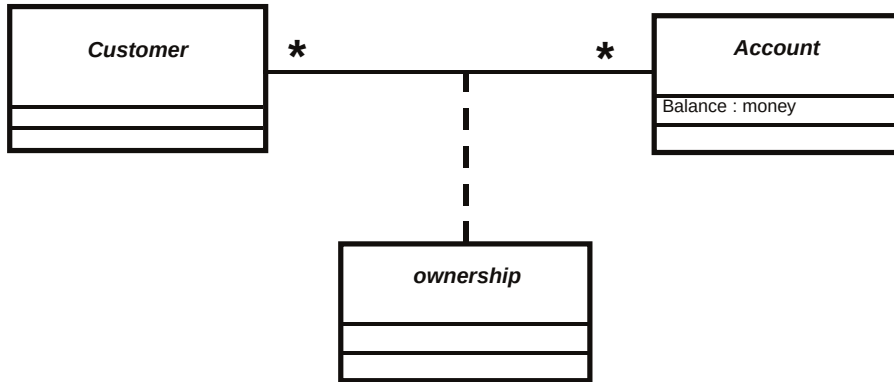


Fig. 4. Making the relationship explicit via an Association class.

From the discussion held above about the disadvantages of the attribute-based representation of the relationship between accounts and customers, it seems clear that the typical implementation of associations through attributes would take us one step back. The best way of implementing the contract through the association class would seem to be for a new operation to be declared for *ownership* that can act as a mediator between the roles. Upon a call from the client, the mediator would have the responsibility of determining whether the contractual relationship between the partners is valid and, in case it is, delegate to the supplier to proceed. In this way, it would, indeed, be possible to achieve the required flexibility for accommodating changes in the business rules simply by modifying the contracts as required, e.g. at the level of the preconditions of the mediators, without having to modify the partners in the contract.

Although the advantages of making relationships first-class citizens in conceptual modelling has been recognised by many authors (e.g. [18]), which lead to the ISO General Relationship Model (ISO/IEC 10165-7), things are not as clean with this solution as they may seem. On the one hand, the fact that a mediator is used for coordinating the interaction between two given objects does not prevent direct relationships from being established that may side step it and violate the contract. In the case of the account package, nothing prevents a designer from connecting directly a customer to

an account through interactions, possibly breaching the contract because the precondition has now been moved from the account to the mediator. On the other hand, the solution is still intrusive in the sense that the calls to the mediator must be explicitly programmed in the implementation of the classes involved in the association. Moreover, it is not incremental in the sense that the addition of new business rules cannot be achieved by simply introducing new association classes and mediators. The other classes in the system need to be made aware that new association classes have become available so that the right mediators are used for establishing the required interactions. That is, the burden of deciding which mediator to interact with is put again on the side of clients. On the other hand, different rules may interact with each other thus requiring an additional level of coordination among the mediators themselves to be programmed. This leads to models that are not as abstract as they ought to be due to the need to make explicit (even program) the relationships that may exist between the original classes and the mediators, and among the different mediators themselves.

The primitive that we have developed for modelling this kind of contractual relationship between components circumvents these problems by relying on a mechanism of superposition that is more powerful than the use of mediators as outlined above. It provides a coordination role that is closer to what is available for configurable

distributed systems and software architectures, in general, as presented in the next sections, with the advantage of being implemented in a way that is non-intrusive and incremental, enabling a smooth evolution through dynamic reconfiguration.

3. Coordination contracts

From a static point of view, a coordination contract defines an *association class* in the sense of the UML (i.e. an association that has all the attributes of a class). However, the way interaction is established between the partners is more powerful: it provides a *coordination role* that is closer to what is available for configurable distributed systems in general, namely through the use of architectural connectors [19]. A contract consists of a collection of role classes (the partners in the contract) and the prescription of the coordination effects (the glue in the terminology of software architectures) that will be superposed on the partners. Coordination contracts take the following general form:

```
contract <name>
  partners <list-of-classes>
  invariant <the relation between the partners>
  constants
  attributes
  operations
  coordination <behaviour superposed by the contract>
  behaviour <local behaviour of the contract>
end contract
```

An example can be given immediately through the account packages already discussed. The traditional package, by which withdrawals require that the balance be greater than the amount being withdrawn, can be specified as follows, where by Account we mean the class specification discussed in the previous section but without the enabling restriction on withdrawals:

```
contract Traditional package
  partners x : Account; y : Customer;
  invariants ?owns(x,y)= TRUE;
  coordination
    tp: when y.calls(x.Withdrawal(z))
        do x.Withdrawal(z)
        with x.Balance() ≥ z;
end contract
```

The instances of the partner classes that can actually become coordinated by instances of the

contract are determined through a set of conditions specified as invariants. The typical case, of which the traditional package is an example, is for instances to be required to belong to some association between the partners (the ownership association in our example).

The behaviour that is required to be superposed over that of the partners is identified under “coordination” as trigger/reaction clauses of the form:

```
<name> : when <condition>
        do <set of actions>
        with <condition>
```

Each coordination clause has a name that can be used for managing the interference between all the clauses and the contract’s own actions, e.g. in terms of precedence relations. This is similar to what happens in parallel program design languages like Interacting Processes [13]. The condition under “when” establishes the trigger of the clause. Typical triggers are the occurrence of actions or state changes in the partners. Under “do” we identify the reactions to be performed, usually in terms of actions of the partners and some of the contract’s own actions, which constitute what we call the *synchronisation set* associated with the trigger. Finally, under “with”, we can put further constraints on the actions involved in the synchronisation set, typically further preconditions. The intuitive semantics (to be further discussed in the following section) is that, through the “when” condition, the contract intercepts calls to the partners or detects events in the partners to which it has to react. It then checks the “with” conditions to determine whether the interaction can proceed and, if so, coordinates the execution of the synchronisation set. All this is done atomically, in a transactional mode, in the sense that either all the actions in the set are executed or none is.

The flexible package involves only one such interaction. It relates calls placed by the customer for withdrawals with the actual withdrawal operation of the corresponding account. The customer is the trigger of the interaction: the interaction requires every call of the customer to synchronise

with the withdrawal operation of the account but enables other withdrawals to occur outside the interactions, e.g. by other joint owners of the same account. The constraint imposed through the with-clause is the guard already discussed in the previous section, which is now externalised as part of the contract. Hence, it regulates only a specific class of interactions between customers and accounts: those that have subscribed to the particular contract. In particular, each instance of the contract and, hence, the constraint, applies only to an identified pair of customer and account, meaning that other owners of the same account may subscribe to different contracts.

The notation involving the interaction in this example is somewhat redundant because the fact that the trigger is a call from the customer to an operation of the account immediately identifies the reaction to be performed. In situations like this, a modelling language should allow for abbreviated syntactical forms. However, in the paper, we will consistently present the full syntax to make explicit the various aspects involved in an interaction. In particular, the full syntax makes it explicit that the call put by the client is intercepted by the contract, and the reaction, which includes the call to the supplier, is coordinated by the contract. Again, we stress that the interactions established through contracts are atomic, i.e. the synchronisation set determined by each coordination entry of the contract is executed as a single transaction—either all the actions in the set are performed or none is. In particular, the client will not know what kind of coordination is being superposed. From the point of view of the client, it is the supplier that is being called.

As already explained, the purpose of contracts is to externalise interactions between objects, making them explicit in the conceptual models, thus reflecting the business rules that apply in the current state. Hence, contracts may change as the business rules change, making system evolution compositional with respect to the evolution of the application domain. For instance, new account packages may be introduced that relax the conditions under which accounts may be

overdrawn:

```

contract VIP package
  partners x : Account; y : Customer;
  constants CONST_VIP_BALANCE: Integer;
  attributes Credit : Integer;
  invariants
    ?owns(x,y)=TRUE;
    x.AverageBalance()>= CONST_VIP_BALANCE;
  coordination
    vp: when y.calls (x.Withdrawal(z))
        do x.Withdrawal(z)
        with x.Balance()+ Credit()≥z;
end contract

```

Notice that, on the one hand, we have strengthened the invariant of the contract, meaning that only a restricted subset of the population can become under the coordination of this new business rule, namely the customers that qualify as VIPs. On the other hand, the contract weakens the guard imposed on withdrawals, meaning that there are now more situations in which customers can withdraw money from their accounts: this is the added benefit of the more restricted class of customers.

In general, we allow for contracts to have features of their own. This is the case of the contract above for which an attribute and a constant were declared to model the credit facility. It is important to stress that such features (including any additional operations) are all private to the contract: they fulfil a local purpose in establishing the coordination that is required between the partners and are not available for interaction with other objects. Indeed, the contract does not define a public class and its instances are not considered as ordinary components of the system. This is one of the reasons why association classes, as available in the UML, are not expressive enough to model the coordination mechanisms of contracts. Although contracts allow for interactions to be made explicit in conceptual models, they should not be accessed in the same way as the classes that model the core business entities. Contracts do not provide services: they coordinate the services made available by the core entities. Contracts are the subject of a different level of management of the business component system: that of the definition and evolution of its configuration.

Another shortcoming of association classes is that they do not enforce the synchronisation and atomicity requirements of the proposed coordination mechanisms. As an example, consider one of

the latest products to appear in banking—“the flexible package”. This is a mechanism via which automatic transfers are made between a checking account and a savings account of the same client: from savings to checking when the balance is not enough to satisfy a withdrawal, and from checking to savings when the balance goes above a certain threshold.

Like before, the application of traditional object-oriented techniques for adding this new feature to the system would probably raise a number of problems. The first one concerns the decision on where to place the code that is going to perform the transfers: the probable choice would be the checking account because that is where the balance is kept. Hence, the implementation of account would have to be changed. The “natural” solution would be to assign the code to a new association class between the two accounts but, again, current techniques for implementing association classes require the implementations of the participating classes to be changed because the associations are implemented via attributes.

Another problem is concerned with the handling of the synchronisation of the transfers. If the transfers are not coded in the methods of the accounts, there is no way in which the whole process can be dealt with atomically as a single transaction. Again, what is required is a mechanism via which we can superpose a layer of coordination that is separate from the computations that are performed locally in the objects and enforces the required interactions, including the synchronisation constraints. This is exactly the level of coordination that contracts provide.

```
contract Flexible package
  partners c, s : Account;
  attributes maximum, minimum : Integer;
  invariants c.owner=s.owner;
  coordination
    putfunds:
      when c.balance()<minimum
      do
        s.debit(min(s.balance(),maximum-c.balance()))
        and c.credit(min(s.balance(),maximum - c.balance()));

    getfunds:
      when c.balance()>maximum
      do
        c.debit(c.balance()-maximum)
        and s.credit (c.balance()-maximum);
end contract
```

We are using this example to illustrate another useful class of triggers: the detection of state changes that occur in the partners, in this case the

balance of the checking account falling below the minimum or rising above the maximum. Naturally, for specific languages and implementation forms, certain forms of triggers may not be available, and another form of trigger would have to be provided to obtain the same effect.

4. Semantical aspects

The intuitive semantics of contracts can be summarised as follows:

- Contracts are added to systems by identifying the instances of the partner classes to which they apply. These instances may belong to subclasses of the partners; for instance, in the case of the flexible package, both partners were identified as being of type *account* but, normally, they will be applied to two subclasses of account: a checking account and a savings account. The actual mechanism of identifying the instances that will instantiate the partners and superposing the contract is outside the scope of the paper. This can be achieved directly as in languages for reconfigurable distributed systems [20], or implicitly by declaring the conditions that define the set of those instances.
- Contracts are superposed on the partners taken as black-boxes: the partners in the contract are not even aware that they are being coordinated by a third party. In a client–supplier mode of interaction, instead of interacting with a mediator that then delegates execution on the supplier, the client calls directly the supplier; however, the contract “intercepts” the call and superposes whatever forms of behaviour are prescribed; this means that it is not possible to bypass the coordination being imposed through the contract because the calls are intercepted.
- The same transparency applies to all other clients of the same supplier: no changes are required on the other interactions that involve either partner in the contract. Hence, contracts may be added, modified or deleted without any need for the partners, or their clients, to be modified as a consequence.

- The interaction clauses in a contract identify points of *rendez-vous* in which actions of the partners and of the contract itself are synchronised; the resulting synchronisation set is guarded by the conjunction of the guards of the actions in the set and the condition indicated in the with-clause. The execution of the synchronisation set requires the execution of all the actions in the set.
- The implementation of the synchronisation sets that are determined by the contracts can either be obtained through the primitives available in the transaction language (e.g. calls to the actions participating in the synchronisation set can be made by the regulator), or a completely new implementation can be superposed by the contract body as an alternative to the implementations available in the partners. In the latter case, the properties of the interfaces have to be preserved, namely any pre/post conditions and invariants specified through contracts in the sense of Meyer's "design by contract" [17]. This is a useful way of bypassing legacy code as pointed out in [7]. Its implementation requires, however, that the contract body (the regulator) be awarded some sort of priority over the roles at the execution level.
- The effect of superposing a contract is cumulative: more than one contract may be active at a given state of the system. Because the superposition of a contract consists, essentially, of synchronous interactions, the different active contracts will superpose their coordinating behaviour, achieving a cumulative effect in the sense that the synchronisation set to be executed is the union of all the synchronisation sets of the active contracts and its guard is given by the conjunction of all the guards and "with" clauses associated with the active contracts. For instance, in the example of the flexible package, the transfers can also be subject to contracts that regulate the discipline of withdrawals for the particular account and client.
- In the paper, we will not address the issue of managing which contracts are active at which states. This aspect is being addressed as part of the configuration language that is being developed for assisting in the process of controlling

or programming the evolution of systems. This will include logical mechanisms for reasoning about possible interactions between contracts. Preliminary work in this direction is reported in [21].

For simplicity, the examples are based on binary relationships. However, contracts may involve more than two partners. In this case, the invariant and coordination clauses may refer to all partners. In fact, contracts may be seen to correspond to synchronisation agents, as presented in the Actors model [22], that coordinate the rules of engagement of various objects participating simultaneously in the same task.

Several authors have already made similar observations about the need to make explicit and available, as first-class citizens, the rules that govern the behaviour of systems, namely in [18], which became the subject matter of the ISO General Relationship Model (ISO/IEC 10165-7). The notion of coordination contract that we propose is in the spirit of this work but adds to it the evolutionary aspects that it inherits from the architectural approaches, and the concurrency and synchronisation aspects that it inherits from the notion of superposition as used for parallel program design (Fig. 5).

Indeed, the intuitive semantics that we have just outlined shows that our approach capitalises on a number of proposals that have been made in the areas of Programming Languages (coordination languages and models), Parallel Program Design (superposition), and Software Engineering (software architectures). In fact, we view the notion of

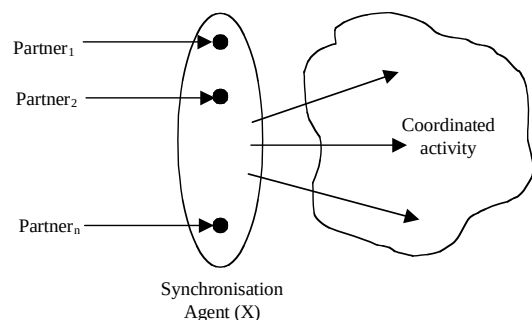


Fig. 5. A Synchronisation Agent.

coordination contract as a synthesis of these three aspects which, together with the experience that we have gathered in information system development, justify an investment in methodologies and tools for supporting contract-based development.

Having acknowledged the sources on which our proposal is grounded we must, nevertheless, emphasise that other approaches can be found in the literature that, in many ways, are similar in spirit or capture some of the intuitions that we had when developing contracts. We should start by saying that the term ‘contract’ itself has been quite overloaded:

- There are, of course, contracts in the sense of Meyer [17]; they fulfil a different, but complementary, role to the coordination contracts that we have described: their purpose is to support the development of object methods in the context of client–supplier relationships between objects. Therefore, they apply, essentially, to the “in-the-small” *construction* of systems rather than the “in-the-large” dimension that we have chosen as target and which is concerned with *architecture* and *evolution*. In fact, in what concerns evolution, the use of Meyer’s contracts in a non-architectural approach leads to some of the problems that we discussed in the introduction: by adopting clientship as the basic discipline for object interconnection, a bias is introduced in the way business rules get coded up, assigning to the supplier side the responsibility for accommodating changes that, from the point of view of the business rules, belong to the client. This was the case of the flexible withdrawals for VIP customers: by placing the contract on the supplier side (the account), the new rules are more easily modelled as specialisations of account whereas, in the application domain, they reflect specialisations of the client.
- A notion of contract that applies to behaviours and not to individual operations or methods is the one developed in [23]. The aim of contracts as developed therein is to model collaboration and behavioural relationships between objects that are jointly required to accomplish some task. The emphasis, however, is in software construction, not so much in evolution.

- A notion of contract can also be found in [24] that emerged in the context of the action-systems approach [11]. Like in our case, it promotes the separation between the specification of what actors can do in a system and how they need to be coordinated so that the computations required of the system are indeed part of the global behaviour. The architectural and evolutionary dimensions are not explored as such.

Besides these related notions of contract, other approaches (e.g. [5,6]) can be found in the literature that explore the use of architectures in evolution. However, they tend to do it at a level of abstraction that misses the business aspects: they support changes operated at the level of the languages and platforms over which components are implemented, but they miss the abstraction power that superposition provides as a general technique for enabling contract (or connector)-based evolution, as well as the setting-up of semantic primitives which, like coordination contracts, can be used in conjunction with more abstract languages for supporting business modelling and evolution.

The intuitive semantics that we outlined above has been given both a mathematical and an implementational counterpart. The mathematical semantics draws on previous work on the categorical semantics of architectural and coordination principles. The presentation of this semantics is outside the scope of this paper. Please consult [25] for the formalisation of different kinds of superposition that lead to the identification of different kinds of contracts (regulators, monitors, etc.); [26] for a formalisation of architectural connectors based on the previous formalisation of superposition, which includes the semantics of instantiation of the partners (roles); [27] for the coordination aspects as related to superposition and the application to software architectures; and [28,29,21] for the application to dynamic reconfiguration, including the definition of algebraic operations on architectural connectors that are directly applicable to contracts.

The fact that a mathematical semantics exists for justifying our principle of supporting evolution

through the superposition of contracts does not mean that it can be put into practice directly over the technology that is available today. In [16,30] we have shown that, even if none of the standards for component-based software development that have emerged in the last few years (e.g. CORBA, EJB and COM) can provide a convenient and abstract way of supporting the proposed coordination principles as first-class mechanisms, an implementation can be given that is based on a micro-architecture that exploits some widely available properties of object-oriented programming languages such as polymorphism and subtyping. The relationship between this micro-architecture and the mathematical semantics is discussed in [31]. A prototype of an environment for supporting development of Java applications with coordination contracts is also available that adopts this micro-architecture [www.atxsoftware.com/CDE].

5. Concluding remarks

In this paper, we have suggested that Information System Engineering can be significantly enhanced by adopting *Coordination Technologies* through which systems can be made more *agile* in responding to the turbulence that characterises current business and technological environments. More specifically, we described how a new semantic primitive—*coordination contracts*—can be added to current object-oriented approaches to fulfil some of the promises of Component-Based Development in terms of supporting a new approach to business and technology architecture in which components can be dynamically added, modified, replaced and reconfigured [1,9,3,2].

Contracts draw from several mechanisms that have been available for sometime in Software Engineering:

- Contract-based development is based on the idea of separating *computation* from *coordination*, i.e. of making clear what components in a system provide the functionalities on which services are based, and what mechanisms are responsible for coordinating the activities of

those components so that the properties that are required of the global behaviour of the system emerge from the interactions that are established. This idea has been promoted by researchers in the area of Programming Languages who have coined the term “Coordination Languages and Models” (e.g. [10]).

- The importance of having these coordination mechanisms available as first-class entities and as units of structure in system models and designs was inspired by the role played by connectors in software architectures [19,32]. This is why we proposed contracts as a semantic primitive enriching the notion of association class.
- The ability for objects to be treated as black-box components and, hence, for contracts to be dynamically added or removed from a system without having to recompile the partners, is achieved through the mechanism of superposition (or superimposition) developed in the area of Parallel Program Design [11–14].

An effective use of the proposed coordination mechanisms further requires language and tool support for the actual run-time superposition of contracts, either in reaction to customer demands or for the enforcement of business policies. Such policies have to be understood as “invariants” that apply to “evolution time” and not “computation time”. That is to say, they constrain the way systems can be evolved in terms of their configuration. Naturally, they will also have an impact on the computations that are performed in the system because, as we have seen, the configuration determines the interactions from which the global computational behaviour of the system emerges. What is important to emphasise is that decisions on which contracts can be subscribed by which classes of partners and in which states of the system, are an integral part of the modelling activity. This is the direction in which we are currently working.

To summarise, it is clear that, by bringing to bear techniques from Coordination Languages and Models, Software Architectures, and Parallel Program Design, the notion of contract-based software evolution that we proposed in this

chapter shares many of the advantages of similar approaches that have been proposed in the literature exploring some of these combinations (but not all of them). The experience that we have gathered in applying the concept in real-world projects suggests that contracts are not only an effective mechanism for enabling a flexible evolution of business products, but also a powerful modelling primitive that analysts find useful at the earlier stages of domain modelling, at least in the business areas that we have already mentioned.

Acknowledgements

We would like to thank J. Gouveia, G. Koutsoukos, A. Lopes and M. Wermelinger for all their contributions to the development of the ideas, semantics, and technologies presented in this article. Ana Moreira has also provided useful feedback on the form and contents of this paper.

References

- [1] P. Finger, Component-based frameworks for e-commerce, *Commun. ACM* 43 (10) (2000) 61–66.
- [2] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, Addison-Wesley, Reading, MA, 1998.
- [3] J. Hopkins, Component primer, *Commun. ACM* 43 (10) (2000) 27–30.
- [4] J. Kramer, Exoskeletal software, in: *Proceedings of the 16th ICSE*, 1994, pp. 366.
- [5] H. Evans, P. Dickman, Zones, contracts and absorbing change: an approach to software evolution, in: *Proceedings of the OOPSLA'99*, ACM Press, New York, 1999, pp. 415–434.
- [6] P. Oreizy, N. Medvidovic, R. Taylor, Architecture-based runtime software evolution, in: *Proceedings of the ICSE'98*, IEEE Computer Science Press, Silver Spring, MD, 1998.
- [7] J. Bosch, Superimposition: a Component Adaptation Technique, in: *Information and Software Technology*, 1999.
- [8] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object Oriented Software*, Addison-Wesley, Reading, MA, 1995.
- [9] P. Herzum, O. Sims, *Business Component Factory*, Wiley, New York, 2000.
- [10] D. Gelernter, N. Carriero, Coordination languages and their significance, *Commun. ACM* 35 (2) (1992) 97–107.
- [11] R. Back, R. Kurki-Suonio, Distributed cooperation with action systems, *ACM TOPLAS* 10 (4) (1988) 513–554.
- [12] K. Chandy, J. Misra, *Parallel Program Design—A Foundation*, Addison-Wesley, Reading, MA, 1988.
- [13] N. Francez, I. Forman, *Interacting Processes*, Addison-Wesley, Reading, MA, 1996.
- [14] S. Katz, ASuperimposition Control Construct for Distributed Systems, *ACM TOPLAS* 15 (2) (1993) 337–356.
- [15] L.F. Andrade, J.L. Fiadeiro, Coordination: the evolutionary dimension, in: W. Pree (Ed.), *Technology of Object-Oriented Languages and Systems—TOOLS 38*, IEEE Computer Society Press, Silver Spring, MD, 2001, pp. 136–147.
- [16] L.F. Andrade, J.L. Fiadeiro, Interconnecting objects via contracts, in: R. France, B. Rumpe (Eds.), *UML'99—Beyond the Standard*, LNCS 1723, Springer, Berlin, 1999, pp. 566–583.
- [17] B. Meyer, *Applying Design by Contract*, IEEE Computer, Silver Spring, MD, October 1992, pp. 40–51.
- [18] H. Kilov, J. Ross, *Information Modeling: an Object-Oriented Approach*, Prentice-Hall, Englewood Cliffs, NJ, 1994.
- [19] R. Allen, D. Garlan, A formal basis for architectural connectors, *ACM TOSEM* 6 (3) (1997) 213–249.
- [20] J. Magee, J. Kramer, Dynamic structure in software architectures, in: *Fourth Symposium on Foundations of Software Engineering*, ACM Press, New York, 1996, pp. 3–14.
- [21] M. Wermelinger, J.L. Fiadeiro, Algebraic software architecture reconfiguration, in: *Software Engineering—ESEC/FSE'99*, LNCS 1687, Springer, Berlin, 1999, pp. 393–409.
- [22] G. Agha, *ACTORS: a Model of Concurrent Computation in Distributed Systems*, MIT Press, Cambridge, MA, 1986.
- [23] R. Helm, I. Holland, D. Gangopadhyay, Contracts: specifying behavioral compositions in object-oriented systems, in: *Proceedings of the OOPSLA'90/ECOOP'90*, ACM Press, New York, 1990, pp. 169–180.
- [24] R.J. Back, L. Petre, I. Paltor, Analysing UML use cases as contracts, in: R. France, B. Rumpe (Eds.), *UML'99—Beyond the Standard*, LNCS 1723, Springer, Berlin, 1999, pp. 518–533.
- [25] J.L. Fiadeiro, T. Maibaum, Categorical semantics of parallel program design, *Sci. Comput. Program.* 28 (1997) 111–138.
- [26] J.L. Fiadeiro, A. Lopes, semantics of architectural connectors, in: *TAPSOFT'97*, LNCS 1214, Springer, Berlin, 1997, pp. 505–519.
- [27] J.L. Fiadeiro, A. Lopes, Algebraic semantics of coordination, or what is in a signature?, in: A. Haeberer (Ed.), *AMAST'98*, Springer, Berlin, 1999.
- [28] M. Wermelinger, J.L. Fiadeiro, Connectors for mobile programs, *IEEE Trans. Software Eng.* 24 (5) (1998) 331–341.
- [29] M. Wermelinger, J.L. Fiadeiro, Towards an algebra of architectural connectors: a case study on synchronisation for mobility, in: *Proceeding of the Ninth International Workshop on Software Specification and Design*, IEEE

- Computer Society Press, Silver Spring, MD, 1998, pp. 135–142.
- [30] J. Gouveia, G. Koutsoukos, L. Andrade, J. Fiadeiro, Tool support for coordination based evolution, in: W. Pree (Ed.), *Technology of Object-Oriented Languages and Systems—TOOLS 38*, IEEE Computer Society Press, Silver Spring, MD, 2001, pp. 184–196.
- [31] L.F. Andrade, J.L. Fiadeiro, J. Gouveia, A. Lopes, M. Wermelinger, Patterns for coordination, in: G. Catalin-Roman, A. Porto (Eds.), *COORDINATION'00*, LNCS 1906, Springer, Berlin, 2000, pp. 317–322.
- [32] D. Perry, A. Wolf, Foundations for the Study of Software Architectures, *ACM SIGSOFT Software Eng Notes* 17 (4) (1992) 40–52.